

On-Device Domain Learning for Keyword Spotting on Low-Power Extreme Edge Embedded Systems

Cristian Cioflan[†] , Lukas Cavigelli[‡] , Manuele Rusci^{||} , Miguel de Prado[¶] , Luca Benini^{†§} 

[†] *Integrated Systems Laboratory, ETH Zurich*; [‡] *Zurich Research Center, Huawei Technologies*;

^{||} *ESAT, KU Leuven*; [¶] *VERSES AI*; [§] *DEI, University of Bologna*;

{cioflan, lbenini}@iis.ee.ethz.ch, lukas.cavigelli@huawei.com, manuele.rusci@esat.kuleuven.be, miguel.deprado@verses.ai

Abstract—Keyword spotting accuracy degrades when neural networks are exposed to noisy environments. On-site adaptation to previously unseen noise is crucial to recovering accuracy loss, and on-device learning is required to ensure that the adaptation process happens entirely on the edge device. In this work, we propose a fully on-device domain adaptation system achieving up to 14% accuracy gains over already-robust keyword spotting models. We enable on-device learning with less than 10 kB of memory, using only 100 labeled utterances to recover 5% accuracy after adapting to the complex speech noise. We demonstrate that domain adaptation can be achieved on ultra-low-power microcontrollers with as little as 806 mJ in only 14 s on always-on, battery-operated devices.

Index Terms—On-Device Learning, Domain Adaptation, Low-Power Microcontrollers, Extreme Edge, TinyML, Noise Robustness, Keyword Spotting

I. INTRODUCTION

Keyword Spotting (KWS) pipelines achieve high classification accuracy in clean, noiseless environments. However, their performance severely decreases (*i.e.*, up to 27% [1]) when the data used to train the model offline, on remote servers, differs from the in-field data, *e.g.* due to different on-site noise conditions [1]. Works addressing this issue often remove the noise from the Deep Neural Network (DNN)’s input signal [2], or they artificially augment utterance samples with noise during the offline training to gain noise-robust acoustic models [3]. Nonetheless, in such scenarios, the devices running KWS in inference mode lack the adaptation mechanisms to specialize the deployed model on noises unseen during DNN training, crucial for highly non-stationary noises.

The envisioned adaptation should happen fully on-board to achieve a privacy-by-design system while also minimizing the communication energy costs [4]. The migration of the Machine Learning (ML) pipeline to low-cost, low-power edge devices—known as the TinyML paradigm—focused initially on inference [5]–[7]. Nonetheless, On-Device Learning (ODL) frameworks have been recently proposed [8]–[10], enabling partial or total DNN updates on embedded platforms in the presence of on-site, real-world data distribution shifts. Although still an open problem due to the tight constraints (*i.e.*, memory, storage, latency, device lifetime) within embedded systems, these preliminary approaches are poised to enable efficient post-deployment adaptation at the extreme edge.

This work demonstrates fully on-device domain adaptation of keyword spotting models in ultra-low-power systems. We

achieve accuracy gains up to 14% compared to an offline-trained, noise-robust (herein named Noise-Aware Keyword Spotting (NA-KWS)) model, in previously unseen noisy conditions. We tackle more difficult scenarios than [1], by considering environments with a Signal-to-Noise Ratio (SNR) of 0 dB. Moreover, we propose memory- and storage-reduction strategies considering resource-constrained ODL for domain adaptation, *i.e.*, On-Device Domain Adaptation (ODDA). We thus improve by 5% the accuracy in challenging *meeting* noise conditions with read-write memory requirements as low as 10 kB. Finally, we are the first to demonstrate in practice the constrained ODDA on an ultra-low power, multi-core GAP9 platform, enabling on-site domain adaptation in only 14 s at modest costs of 384 μ J per sample.

II. RELATED WORK

Following the taxonomy introduced by Lopez-Espejo *et al.* [11], environmental noise can be addressed through front-end or back-end methods. The former refers to strategies such as Active Noise Cancellation (ANC) [12], [13] or Speech Enhancement (SE) [2], [14], which generate clean speech signal by subtracting the estimated noise component from the input signal before reaching the acoustic model. Front-end modules increase the latency and dimension of KWS systems and do not support post-deployment model fine-tuning.

Alternatively, back-end methods [3], [15], [16] train acoustic models with clean utterances augmented with distortions estimated to be present in the test environment. Such networks mitigate the front-end cost and enhance the classification performance in adverse conditions, yet they lack environmental awareness and fail to specialize on the target noise.

ODL is therefore necessary to achieve TinyML systems specialized on real-world, on-site scenarios while respecting the privacy and security concerns of the users. In the context of keyword spotting, ODL has been studied addressing few-shot learning, targeting smartphones and smartwatches [17], single-board computers [18], or Microcontroller Units (MCUs) [19], [20]. Cioflan *et al.* [1] proposed a domain adaptation methodology for KWS applications. They achieve a high energy cost of 5.81 J, which would considerably reduce the lifetime of always-on, battery-operated devices, while no on-device deployment is demonstrated.

The low number of ODL KWS works is correlated with the few ODL frameworks targeting MCUs. Ren *et al.* [8]

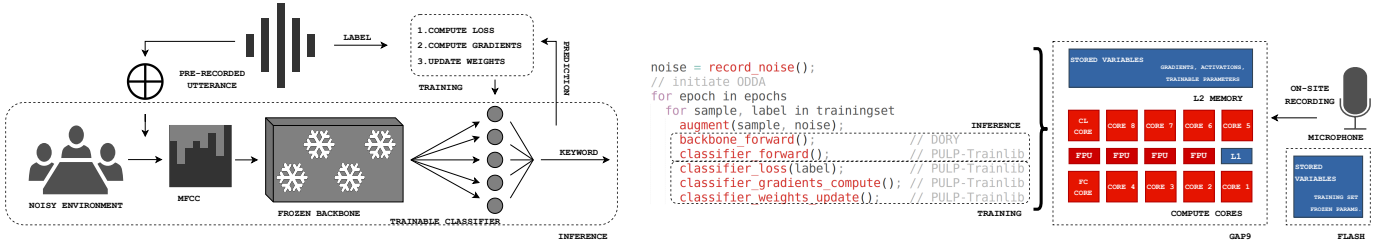


Fig. 1. Overview of the proposed ODDA pipeline. During inference, the on-site recording is processed and its cepstral coefficients are computed, which are then fed to a frozen backbone. The classifier is used to predict the uttered keyword. During training, the on-site recording augments a prerecorded utterance. The prediction and the ground truth are jointly used to update the classifier parameters.

propose a TinyML training solution expanding an already deployed, frozen DNN with a layer that can be customized through Stochastic Gradient Descent (SGD). [9] introduces the *Tiny Training Engine*, generating a backward computation graph enabling sparse tensor update and operator reordering on single-core ARM MCUs. Nadalini *et al.* [10] propose PULP-TrainLib, performing matrix multiplication $36.6\times$ faster than previous single-core solutions and thus accelerating ODL. However, none of the previous works addressed ODL KWS for on-site noise conditions. This paper demonstrates the first on-device domain adaptation work for KWS on ultra-low-power platforms, tackling more difficult scenarios and tighter resource constraints than previous works.

III. BACKGROUND

We consider the ODDA methodology proposed by Cioflan *et al.* [1], refining a KWS network on on-site, previously unseen background noise. A NA-KWS model, which was trained on the server using a full dataset of keyword clean utterances, augmented with multiple noise types, is deployed on the target platform. During inference, the KWS system processes incoming utterances recorded on-site. The signal is preprocessed and fed to the DNN, which predicts the uttered keyword, as shown in Fig. 1 (left).

The ODDA process initiates once a new noise is present in the environment, noise that can potentially harm the performance of the KWS model. First, the noise signal is recorded and used to augment clean, labeled utterances. The utterances belong to the NA-KWS training set and are stored in a non-volatile memory to relieve the user from the tedious process of manually recording (and labeling) data. The resulting signal is fed to the model to compute the prediction and, together with the input label, we determine the cross-entropy loss. We calculate the gradient of the loss with respect to the weights using the backpropagation algorithm and, finally, we update the model weights using Gradient Descent (GD). The steps are repeated for each prerecorded training utterance stored on the board, for multiple adaptation epochs.

IV. RESOURCE-CONSTRAINED ODDA

We refine the ODDA methodology to make it suitable for TinyML settings. Considering always-on, battery-powered target devices, we address energy consumption, which is proportional to the model's adaptation latency (t_{ODDA}), the number

of training samples, and the number of epochs, according to the relation:

$$t_{\text{ODDA}} \propto (t_{\text{infer}} + t_{\text{backprop}}) * \frac{S_{\text{dataset}}}{S_{\text{batch}}} * N_{\text{epochs}}, \quad (1)$$

where S_{dataset} represents the on-device dataset size and S_{batch} , the batch size. We denote as t_{infer} the batch inference latency and as t_{backprop} the latency in updating the model's parameters. The adaptation latency of ODDA depends on the number of noise adaptation rounds (*i.e.*, N_{epochs}), as well as on the target model; a lightweight model, with few Floating-Point Operations (FLOPs)¹ completes the parameter update faster than a complex network.

Another constraint associated with extreme edge devices is the available read-write, on-chip memory. ODDA relies on GD learning, which requires the model inputs, parameters, activations, and gradients, associated with a particular layer, to be available simultaneously in the memory. Although the latency scales inversely with the batch size, as shown in Eq. (1), processing more samples in parallel is limited by the size of the memory containing the backpropagation-associated variables. Lastly, we also address the storage constraints, *i.e.*, the read-only, off-chip memory used to store the model frozen parameters, as well as the prerecorded utterances.

We propose two optimization strategies to reduce the impact of ODDA on the limited available resources. We first enable flexible adaptation depths, partially freezing trainable parameters of the target model. This reduces the number of parameters, activations, and gradients needed for adaptation, thus minimizing the on-chip memory requirements, while simultaneously reducing the computational requirements associated with gradient computation and weight update. Second, we address the available storage space by performing ODDA on a fraction of the keyword samples available during training time. The storage constraints also impact the adaptation time, as performing the same number of adaptation rounds on fewer samples (*i.e.*, reducing S_{dataset} in Eq. (1)) results in fewer FLOPs and, therefore, in a faster learning process.

V. ON-DEVICE IMPLEMENTATION

We implement the ODDA methodology on the GAP9 MCU, depicted in Fig. 1 (right), featuring ten general-purpose RISC-V cores organized in a fabric controller and a nine-core cluster

¹One FLOP represents one floating-point addition or multiplication.

TABLE I
THE EFFECT ON THE ACCURACY [%] OF INCREASING THE TASK COMPLEXITY BY INCREASING THE NUMBER OF CLASSES IN CONJUNCTION WITH AN INCREASE IN MODEL CAPACITY.

Noise Type → Model	Mode	Cafeteria			Meeting			Restaurant			Washing			Metro		
		6 cls.	12 cls.	35 cls.	6 cls.	12 cls.	35 cls.	6 cls.	12 cls.	35 cls.	6 cls.	12 cls.	35 cls.	6 cls.	12 cls.	35 cls.
S	NA-KWS	88.07	78.04	55.60	79.35	65.69	49.19	83.30	73.52	53.68	94.98	89.30	64.94	92.84	83.82	63.89
	ODDA	90.07	80.59	57.31	88.41	78.53	57.19	85.59	76.22	55.97	96.24	90.45	65.47	93.10	85.50	64.95
M	NA-KWS	90.29	82.60	59.60	83.18	69.96	51.06	85.61	77.83	57.03	96.63	91.64	67.58	94.30	87.10	65.82
	ODDA	91.78	84.25	60.69	91.41	84.23	57.87	88.76	80.12	59.01	97.10	92.86	68.15	94.15	89.51	66.66
L	NA-KWS	91.31	83.91	62.16	85.13	70.76	55.27	87.39	79.53	59.31	96.82	92.92	71.46	94.75	87.73	68.40
	ODDA	92.80	85.54	63.53	92.89	84.74	63.49	88.88	81.55	61.19	97.22	93.39	71.97	95.54	90.06	69.10

supporting 8-bit integer (*i.e.*, `int8`) Multiply-Accumulate (MAC) instructions. The cluster also shares four Floating Point Units (FPUs), for half (*i.e.*, `fp16`) and single (*i.e.*, `fp32`) precision operations, and a hardware convolution accelerator, with energy consumption levels as low as 5 μ J for DNNs [21]. GAP9 features a hierarchical memory architecture, with an L1 TCDM of 128 kB, an L2 SRAM of 1.5 MB, and both on- and off-chip L3 Flash and RAM memories.

We use Depthwise Separable Convolutional Neural Network (DS-CNN) as our acoustic model [22]–[24], shown to obtain a competitive keyword spotting system that can function in low- and ultra-low-power platforms. The frozen section of the DS-CNN model is deployed on-device using the DORY code-generator [7], after quantizing its corresponding parameters to `int8`. We moreover store in the off-chip L3 memory prerecorded, labeled utterances used for training NA-KWS. We use PULP-TrainLib [10] to generate inference and training C code for our target, learnable layer(s), computed in `fp32` to preserve the numerical accuracy needed during training.

The ODL algorithm, shown in Fig. 1 (center), initiates in the presence of a new noise. Environmental noise is recorded using a microphone, and it is used to augment the prerecorded training samples. `fp16` Mel-Frequency Cepstral Coefficients (MFCCs) are computed and fed to the frozen backbone. Its dequantized output is employed, together with the utterance label, by the `fp32` classifier to compute the cross-entropy loss. The loss gradients with respect to the weights are then calculated, followed by updating the model’s learnable parameters. The ODDA algorithm therefore requires minimal effort from the end-user, while simultaneously yielding specialized KWS systems ready to be used in noisy environments.

VI. RESULTS

We consider the Google Speech Commands (GSC) [25] dataset as our keyword spotting task, using the data split proposed by the authors. To model the noisy environment, we consider additive noise sourced from real-world recordings of Diverse Environments Multichannel Acoustic Noise Database (DEMAND) [26] dataset. Following Section IV, for NA-KWS, we use, with equal probability, 17 out of 18 noises to augment the samples, together with the absence of any noise, *i.e.*, the *clean* environment. The omitted noise, herein named *target*, is present only after deployment, as opposed to the *source* noises used for offline training. Our training and evaluation

experiments are performed using an SNR of 0 dB, with the DS-CNN models described in Table II.

A. Specializing on noise

In Table I, we analyze the ODDA impact given the task complexity and the model capacity, demonstrating that ODDA enables specialization on any previously unseen noise. The impact of ODDA varies across target environments, with improvements of 1.15% on noises with stationary properties (*i.e.*, *washing*), as the generally-robust NA-KWS network is already capable of determining the speech features in the presence of these noises. On the other hand, domain adaptation is imperative for non-stationary noises, such as *meeting*; ODDA DS-CNN S outperforms by 12% its NA-KWS counterpart in these conditions. Improvements as high as 15% are measured for larger DS-CNNs, leading to top-1 accuracies up to 93% in *washing* conditions. Notably, in the presence of unseen speech noise, domain adaptation is crucial, as a refined DS-CNN S outperforms the $17\times$ larger NA-KWS DS-CNN L by 8%.

When the task complexity increases to a 35-class problem, ODDA brings accuracy improvements between 1% (*i.e.*, *washing*) and 8% (*i.e.*, *meeting*) over NA-KWS models, noise exposure enabling networks to identify distinctive speech properties. We further define a 6-word task (*i.e.*, “yes”, “down”, “left”, “right”, “off”, “stop”) following [27], eliminating short, phonetically similar words (*e.g.* “no” and “go”). The ODDA impact reduces with the task complexity, its benefits peaking at 11% as it is easier to obtain feature separability even in noisy conditions. The ODDA gains lower with the increase in model capacity, down to 7% for DS-CNN L, as larger models are already capable of encoding richer class representations. This comes with unprecedented top-1 accuracies, with the largest models scoring between 88% and 97% in noisy conditions.

B. Resource-constrained ODDA

In Fig. 2 (left) we represent the storage-latency trade-offs for the optimization strategies proposed in Section IV. We focus on speech noise scenarios (*i.e.*, *meeting*), where ODDA is essential. We consider the number of GSC12 samples stored, requiring 32 kB of storage space per sample. With only 3 MB of data, the adapted DS-CNN S (*i.e.*, 26 kB) model surpasses its NA-KWS counterpart by 6%, achieving comparable results with the largest NA-KWS DS-CNN L. When 10% of the data becomes available (*i.e.*, 120 MB – 310 samples per class), ODDA DS-CNN S outperforms the largest noise-robust

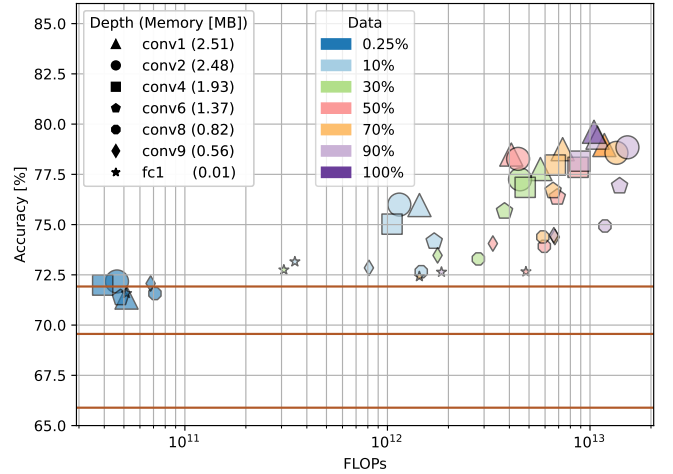
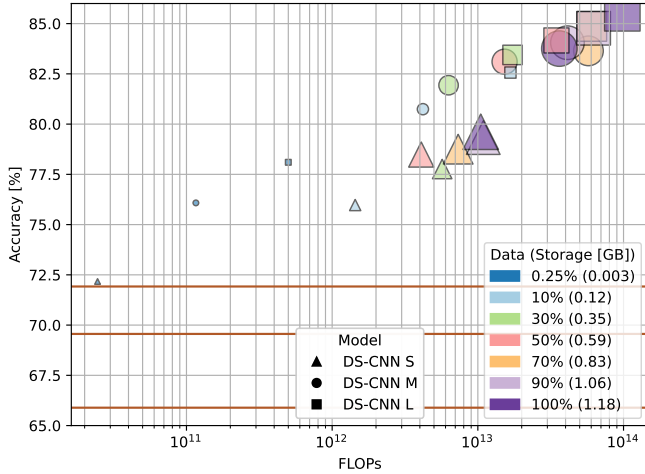


Fig. 2. Mitigating resource constraints during ODDA considering *speech* noise. (left) represents the storage cost given the DS-CNN model (*i.e.*, S, M, L) and the amount of randomly selected data used, while (right) shows the memory cost given the number of updated layers of DS-CNN S, starting with the last linear (*fc1*) layer, for a batch size of two. The horizontal brown lines represent, starting with the lowest one, the baseline accuracy obtained with NA-KWS DS-CNN S, M, and L.

TABLE II

ON-DEVICE DOMAIN ADAPTATION COST PER SAMPLE IN LOW-POWER MODE (LPM; 240 MHz, 650 mV) AND HIGH-PERFORMANCE MODE (HPM; 370 MHz, 800 mV), FOR THREE DS-CNN MODELS. MEMORY REFERS TO THE READ-WRITE MEMORY REQUIREMENTS.

DS-CNN Model	Compute [MFLOPs]	Params. [kB]	Mem. [kB]	Eff.[FLOP/cycle]	Mode	Compute time [ms]	Energy [μJ]
S	2.95	23.7	9.5	4.94	LPM	10.89	424
					HPM	6.74	384
M	17.2	138.1	25.5	9.18	LPM	24.16	988
					HPM	16.34	974
L	51.1	416.7	40.9	11	LPM	55.04	2313
					HPM	32.95	2028

network by 10%, whereas employing the complete training set (*i.e.*, 1.1 GB for 37000 samples) generates accuracy gains up to 14%, demonstrating the positive impact of on-site noise-augmented, prerecorded utterances in ODDA. The model size and the amount of data used also impacts the total number of FLOPs performed during ODDA (*i.e.*, from 1.43 TFLOPs with 10% of the data with DS-CNN S to 10.46 TFLOPs when all the samples are used, and up to 98.10 TFLOPs in the same scenario for DS-CNN L). Although larger models achieve higher accuracies, their complexity increases the inference latency and slows down adaptation.

To address the memory requirements, we only update the last k layers of the model, depicted in Fig. 2 (right). Notably, we outperform the NA-KWS DS-CNN S baseline by 5.5% by refining only its last linear layer (*i.e.*, *fc1*), achieving comparable accuracies with NA-KWS DS-CNN L. This requires storing 780 trainable parameters, the input and output activations, and the weight gradients, *i.e.*, no more than 10 kB of on-chip memory. The accuracy increases with the adaptation depth and the number of utterances employed. Using 10% of available data, refining the entire network outperforms by 1.2% a model of which only the classifier was adapted, whereas the difference increases to 6% for 100% of training samples.

C. GAP9 implementation

Motivated by the memory- and storage-constrained ODDA validated in Section VI-B, we implement the lightweight

domain adaptation on GAP9 MCU. We consider 100 labeled utterances for adapting our KWS system, as described in Section V. In Table II, we depict the ODDA cost per sample on GAP9 MCU for Low-Power Mode (LPM) and High-Performance Mode (HPM) scenarios for the complete processing pipeline. In LPM we measure an average power consumption of 36 mW, saving energy costs during inference, triggered every 50 ms in always-on, battery-operated devices, while HPM enables energy-efficient training under 55 mW.

We refine the classifier layer of DS-CNN S using 100 samples (*i.e.*, ten samples per target class) for 21 epochs, increasing the model accuracy with 6% in only 14 s. In HPM, only 806 mJ are sufficient for effective ODDA. As larger KWS are used, memory requirements concerning classifier update increase from 10 kB to 40 kB, whereas the average energy consumption increases 5.3×; up to 1.9 min are needed to update the largest DS-CNN in HPM.

VII. CONCLUSION

This work proposes a domain adaptation strategy addressing resource-constrained KWS systems. We achieve accuracy gains up to 14% for complex speech noises in 12-word problems. For such speech background noise, only 100 prerecorded, labeled samples are sufficient to reach on-device adaptation by refining the DNN's classifier, with only 3 MB of read-only memory and 10 kB of read-write memory. Notably, we show that a small DS-CNN refined through ODDA outperforms 17× larger networks by up to 8%. We demonstrate our resource-constrained ODDA on GAP9 MCU, achieving a 6% accuracy increase in noisy environments after only 14 s while requiring as little as 806 mJ.

VIII. ACKNOWLEDGEMENTS

This work was partly supported by the Swiss National Science Foundation under grant No 207913: TinyTrainer: On-chip Training for TinyML devices.

REFERENCES

- [1] C. Cioflan, L. Cavigelli, M. Rusci, M. De Prado, and L. Benini, "Towards on-device domain adaptation for noise-robust keyword spotting," in *2022 IEEE 4th International Conference on Artificial Intelligence Circuits and Systems (AICAS)*, 2022, pp. 82–85.
- [2] M. Rusci, M. Fariselli, M. Croome, F. Paci, and E. Flamand, "Accelerating rnn-based speech enhancement on a multi-core mcu with mixed fp16-int8 post-training quantization," in *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*. Springer, 2022, pp. 606–617.
- [3] I. López-Espejo, Z.-H. Tan, and J. Jensen, "A novel loss function and training strategy for noise-robust keyword spotting," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 29, pp. 2254–2266, 2021.
- [4] J. Chen and X. Ran, "Deep learning with edge computing: A review," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1655–1674, 2019.
- [5] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, M. Cowan, H. Shen, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy, "Tvm: An automated end-to-end optimizing compiler for deep learning," in *Proceedings of the 13th USENIX Conference on Operating Systems Design and Implementation*, ser. OSDI'18. USA: USENIX Association, 2018, p. 579–594.
- [6] R. David, J. Duke, A. Jain, V. Janapa Reddi, N. Jeffries, J. Li, N. Kreeger, I. Nappier, M. Natraj, T. Wang, P. Warden, and R. Rhodes, "Tensorflow lite micro: Embedded machine learning for tinyml systems," in *Proceedings of Machine Learning and Systems*, A. Smola, A. Dimakis, and I. Stoica, Eds., vol. 3, 2021, pp. 800–811. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2021/file/6c44dc73014d66ba49b28d483a8f8b0d-Paper.pdf
- [7] A. Burrello, A. Garofalo, N. Bruschi, G. Tagliavini, D. Rossi, and F. Conti, "Dory: Automatic end-to-end deployment of real-world dnns on low-cost iot mcus," *IEEE Transactions on Computers*, pp. 1–1, 2021.
- [8] H. Ren, D. Anicic, and T. A. Runkler, "Tinyol: Tinyml with online-learning on microcontrollers," in *2021 International Joint Conference on Neural Networks (IJCNN)*, 2021, pp. 1–8.
- [9] J. Lin, L. Zhu, W.-M. Chen, W.-C. Wang, C. Gan, and S. Han, "On-device training under 256kb memory," in *Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2022.
- [10] D. Nadalini, M. Rusci, G. Tagliavini, L. Ravaglia, L. Benini, and F. Conti, "Pulp-trainlib: Enabling on-device training for risc-v multi-core mcus through performance-driven autotuning," in *Embedded Computer Systems: Architectures, Modeling, and Simulation*, A. Orailoglu, M. Reichenbach, and M. Jung, Eds. Cham: Springer International Publishing, 2022, pp. 200–216.
- [11] I. López-Espejo, Z.-H. Tan, J. H. L. Hansen, and J. Jensen, "Deep spoken keyword spotting: An overview," *IEEE Access*, vol. 10, pp. 4169–4199, 2022.
- [12] Y. Huang, T. Hughes, T. Z. Shabestary, and T. Applebaum, "Supervised noise reduction for multichannel keyword spotting," in *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2018, pp. 5474–5478.
- [13] Y. A. Huang, T. Z. Shabestary, and A. Gruenstein, "Hotword cleaner: Dual-microphone adaptive noise cancellation with deferred filter coefficients for robust keyword spotting," in *ICASSP 2019 - 2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2019, pp. 6346–6350.
- [14] M. Jung, Y. Jung, J. Goo, and H. Kim, "Multi-task network for noise-robust keyword spotting and speaker verification using ctc-based soft vad and global query attention," in *Interspeech*, 10 2020, pp. 931–935.
- [15] D. Ng, J. Q. Yip, T. Surana, Z. Yang, C. Zhang, Y. Ma, C. Ni, E. S. Chng, and B. Ma, "I2cr: Improving noise robustness on keyword spotting using inter-intra contrastive regularization," in *2022 Asia-Pacific Signal and Information Processing Association Annual Summit and Conference (APSIPA ASC)*, 2022, pp. 605–611.
- [16] D. Ng, Y. Chen, B. Tian, Q. Fu, and E. S. Chng, "Convmixer: Feature interactive convolution with curriculum learning for small footprint and noisy far-field keyword spotting," in *IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP 2022, Virtual and Singapore, 23-27 May 2022*. IEEE, 2022, pp. 3603–3607. [Online]. Available: <https://doi.org/10.1109/ICASSP43922.2022.9747025>
- [17] C. Jagmohan, Y. D. Kwon, and C. Mascolo, "Exploring on-device learning using few shots for audio classification," in *Proc. IEEE EUSIPCO*, 08 2022, pp. 424–428.
- [18] S. Disabato and M. Roveri, "Incremental on-device tiny machine learning," in *Proceedings of the 2nd International Workshop on Challenges in Artificial Intelligence and Machine Learning for Internet of Things*, ser. AIChallengeIoT '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 7–13. [Online]. Available: <https://doi.org/10.1145/3417313.3429378>
- [19] C. Profentzas, M. Almgren, and O. Landsiedel, "Minilearn: On-device learning for low-power iot devices," in *Proc. ACM Int. Conf. on Embedded Wireless Systems and Networks*, ser. EWSN '22. New York, NY, USA: Association for Computing Machinery, 2023, p. 1–11.
- [20] M. Rusci and T. Tuytelaars, "On-device customization of tiny deep learning models for keyword spotting with few examples," *IEEE Micro*, vol. 43, no. 06, pp. 50–57, nov 2023.
- [21] C. Banbury, V. J. Reddi, P. Torelli, J. Holleman, N. Jeffries, C. Kiraly, P. Montino, D. Kanter, S. Ahmed, D. Pau *et al.*, "Mlperf tiny benchmark," *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.
- [22] Y. Zhang, N. Suda, L. Lai, and V. Chandra, "Hello edge: Keyword spotting on microcontrollers," 2017. [Online]. Available: <https://arxiv.org/abs/1711.07128>
- [23] Y. Lu, W. Shan, and J. Xu, "A depthwise separable convolution neural network for small-footprint keyword spotting using approximate mac unit and streaming convolution reuse," in *2019 IEEE Asia Pacific Conference on Circuits and Systems (APCCAS)*, 2019, pp. 309–312.
- [24] P. Sørensen, B. Epp, and T. May, "A depthwise separable convolutional neural network for keyword spotting on an embedded system," *EURASIP Journal on Audio, Speech, and Music Processing*, vol. 2020, 06 2020.
- [25] P. Warden, "Speech commands: A dataset for limited-vocabulary speech recognition," 2018. [Online]. Available: <https://arxiv.org/abs/1804.03209>
- [26] J. Thiemann, N. Ito, and E. Vincent, "DEMAND: a collection of multi-channel recordings of acoustic noise in diverse environments," Jun. 2013, Supported by Inria under the Associate Team Program VERSAMUS. [Online]. Available: <https://doi.org/10.5281/zenodo.1227121>
- [27] M. Mazumder, S. Chitlangia, C. Banbury, Y. Kang, J. M. Ciro, K. Achorn, D. Galvez, M. Sabini, P. Mattson, D. Kanter, G. Diamos, P. Warden, J. Meyer, and V. J. Reddi, "Multilingual spoken words corpus," in *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*, 2021. [Online]. Available: <https://openreview.net/forum?id=c20jiJ5K2H>